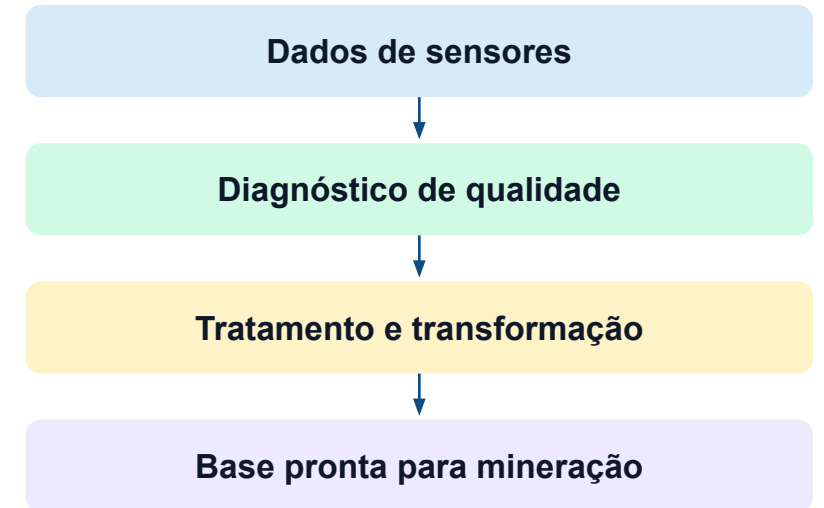


Pré-processamento de dados em IoT

Limpeza, ruídos, valores ausentes, transformação e padronização

Disciplina: Mineração de dados e internet das coisas

Roteiro visual da aula



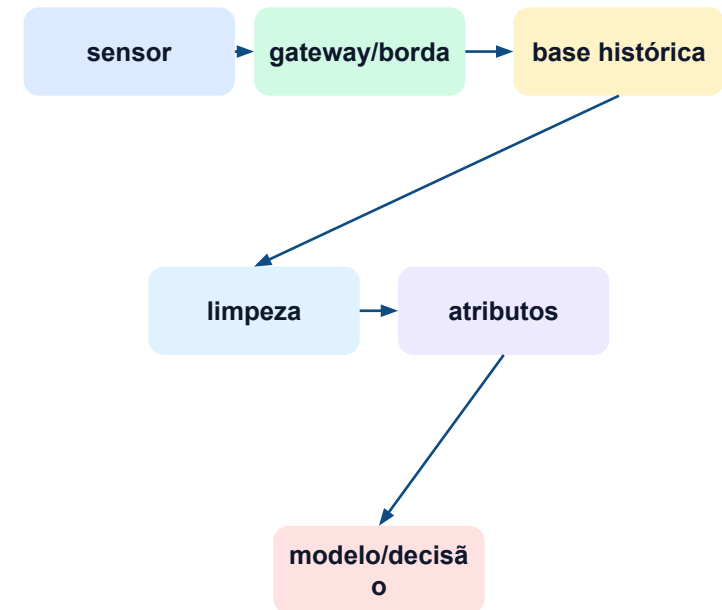
Por que pré-processar dados de IoT?

Tópico: Motivação

Sensores registram fenômenos do mundo físico, mas os dados podem chegar incompletos, atrasados, duplicados ou com unidades diferentes. Antes de aplicar mineração de dados, precisamos preparar a base para que os padrões encontrados representem o comportamento real, e não problemas de coleta.

[Link útil: NIST IoT](#)

IoT: do sensor ao modelo



Objetivos de aprendizagem

Tópico: O que o estudante deverá compreender

Ao final da aula, o aluno deverá reconhecer problemas comuns em bases de sensores, escolher estratégias simples de limpeza, tratar ruídos e valores ausentes, transformar variáveis temporais e aplicar normalização ou padronização antes de uma análise exploratória ou modelo de mineração.

Objetivos

- ✓ identificar problemas
- ✓ tratar ausentes
- ✓ reduzir ruídos
- ✓ criar atributos
- ✓ padronizar escalas

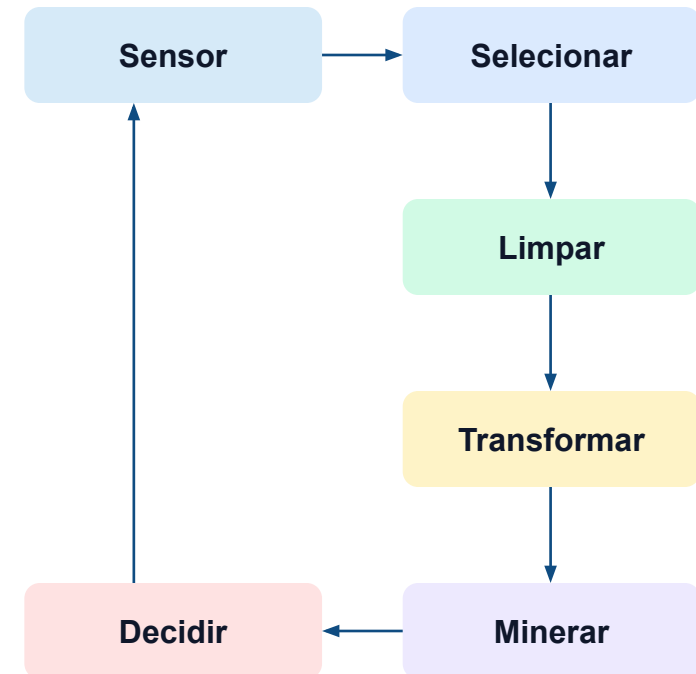
Pré-processamento dentro do KDD

Tópico: Etapa essencial do processo

No KDD, a mineração de dados é apenas uma parte do processo. Antes dela, selecionamos fontes, entendemos o problema, limpamos registros e transformamos atributos. Em IoT, essa etapa é ainda mais importante porque sensores operam continuamente e podem falhar em campo.

[Link útil: KDD clássico](#)

KDD + IoT



O processo é iterativo: uma descoberta pode exigir nova limpeza ou novos atributos.

Dados de sensores: valor com contexto

Tópico: Anatomia do registro

Um valor de sensor não deve ser analisado isoladamente. A leitura precisa de timestamp, unidade, identificador do dispositivo, local, qualidade e condição de operação. Sem esse contexto, é difícil comparar registros, reconstruir eventos e decidir se uma leitura é confiável.

Dados antes da limpeza

hora	temp	umid	status
08:00	23.1	62	ok
08:05	?	63	falha
08:10	240	90	alerta
08:10	240	90	dup.

ausência, valor fora de escala e duplicidade devem ser investigados

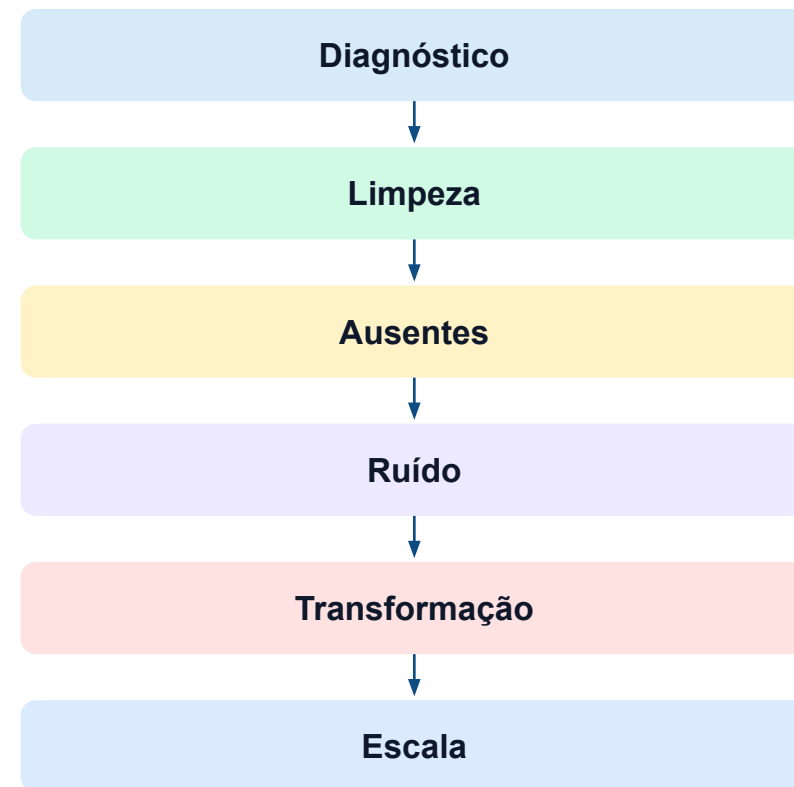
Pipeline de preparação

Tópico: Visão geral das etapas

Um pipeline de pré-processamento organiza as decisões em sequência: diagnosticar a base, corrigir formatos, tratar ausências, remover duplicidades, validar faixas, suavizar ruído, criar atributos e ajustar escalas. Essa sequência deve ser documentada para permitir reprodução dos resultados.

[Link útil: Scikit-learn preprocessing](#)

Pipeline de preparação



Qualidade dos dados

Tópico: Critérios antes do algoritmo

Dados de boa qualidade devem ser completos, consistentes, válidos, atuais e relevantes para o problema. Em IoT, também avaliamos frequência de amostragem, atraso de chegada, sincronização de relógios e estabilidade do sensor. A análise começa medindo a qualidade da própria base.

Qualidade

- ✓ completude
- ✓ consistência
- ✓ validade
- ✓ atualidade
- ✓ relevância
- ✓ sincronização

Limpeza de dados

Tópico: Corrigir problemas visíveis

Limpeza significa identificar registros que prejudicam a análise: duplicados, datas fora de ordem, unidades misturadas, valores impossíveis, códigos de erro e campos inconsistentes. O objetivo não é deixar a base bonita, mas torná-la coerente com o fenômeno medido.

Dados antes da limpeza

hora	temp	umid	status
08:00	23.1	62	ok
08:05	?	63	falha
08:10	240	90	alerta
08:10	240	90	dup.

ausência, valor fora de escala e duplicidade devem ser investigados

Valores ausentes

Tópico: Quando faltam leituras

Valores ausentes aparecem quando o sensor falha, a rede cai, a bateria acaba ou o sistema não grava o registro. Em vez de preencher tudo automaticamente, é necessário investigar o motivo da ausência, pois a falta de dado também pode indicar um evento relevante.

[Link útil: Pandas missing data](#)

Dados antes da limpeza

hora	temp	umid	status
08:00	23.1	62	ok
08:05	?	63	falha
08:10	240	90	alerta
08:10	240	90	dup.

ausência, valor fora de escala e duplicidade devem ser investigados

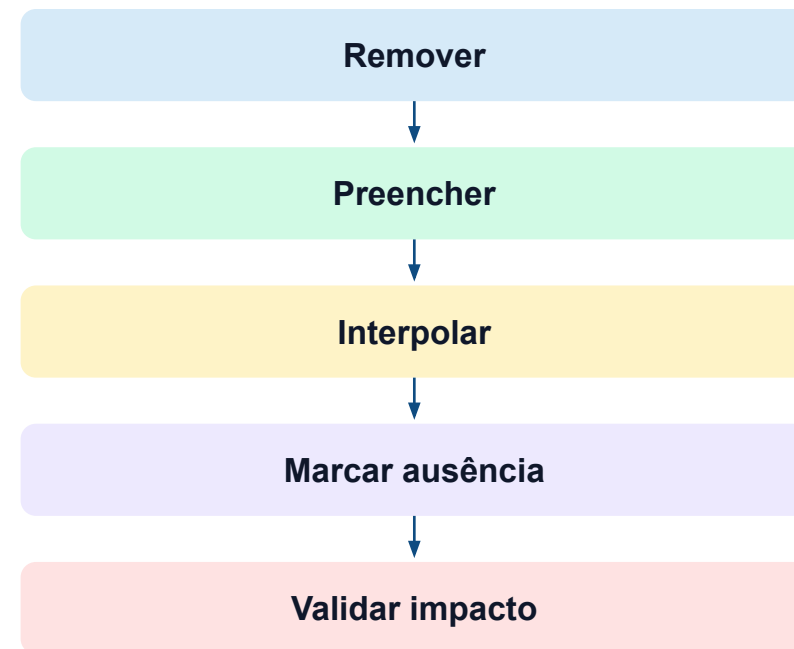
Estratégias para ausentes

Tópico: Remover, preencher ou marcar

As principais estratégias são remover registros, preencher com média ou mediana, repetir o último valor válido, interpolar pela sequência temporal ou criar uma marcação de ausência. Em séries temporais, a escolha deve respeitar o comportamento do fenômeno e a finalidade da análise.

[Link útil: pandas fillna/interpolate](#)

Ausentes

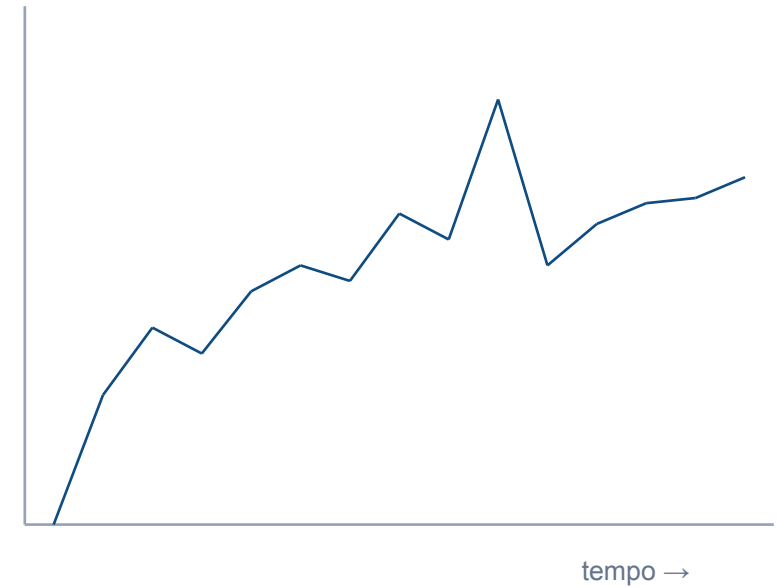


Ruído nos dados

Tópico: Variações indesejadas

Ruído é uma variação indesejada na leitura, causada por interferência, vibração, oscilação elétrica, ambiente instável ou limitação do sensor. Em mineração de dados, ruído pode criar padrões falsos, aumentar erros do modelo e dificultar a interpretação dos gráficos.

Ruído e picos

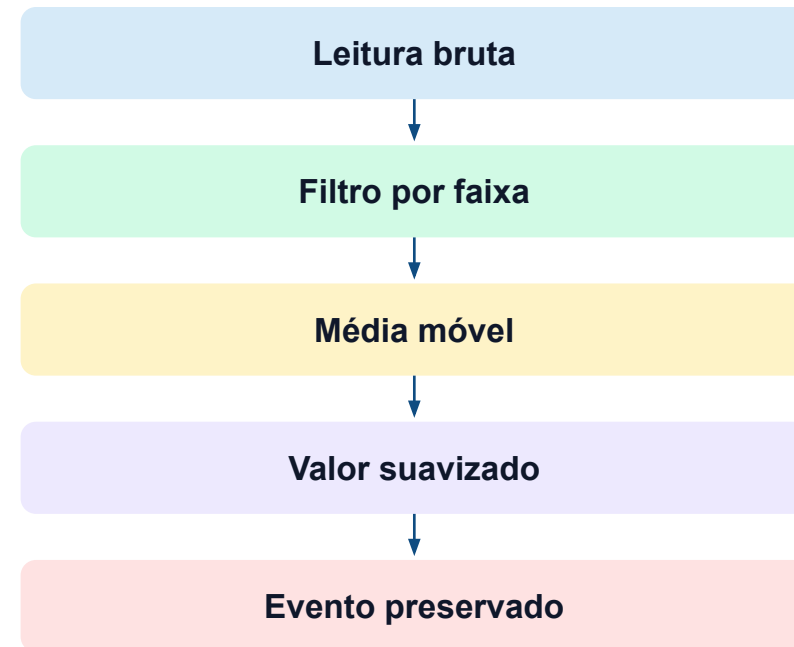


Tratamento de ruído

Tópico: Filtros simples

Para reduzir ruído, podemos aplicar média móvel, mediana móvel, suavização exponencial ou validação por faixa aceitável. Em IoT, filtros podem ser executados na borda para reduzir tráfego, mas é importante não remover picos que representem eventos críticos.

Filtro simples

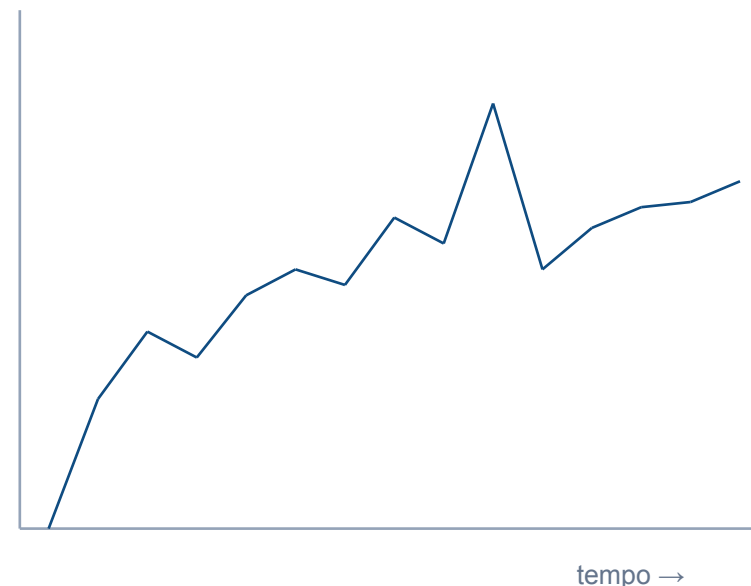


Outliers e anomalias

Tópico: Valor extremo não é sempre erro

Outlier é um valor muito distante do padrão observado. Ele pode ser erro de sensor, falha de comunicação ou evento raro importante. Em IoT, antes de excluir um outlier, devemos verificar horário, unidade, sensor de origem e registros próximos.

Outlier ou evento?



Duplicidades e inconsistências

Tópico: Registros repetidos e conflitos

Duplicidades podem ocorrer quando o gateway reenvia mensagens após falha, quando há reconexão ou quando o processo de ingestão grava o mesmo pacote mais de uma vez. Inconsistências aparecem em nomes de sensores, unidades, categorias e códigos de estado.

Dados antes da limpeza

hora	temp	umid	status
08:00	23.1	62	ok
08:05	?	63	falha
08:10	240	90	alerta
08:10	240	90	dup.

ausência, valor fora de escala e duplicidade devem ser investigados

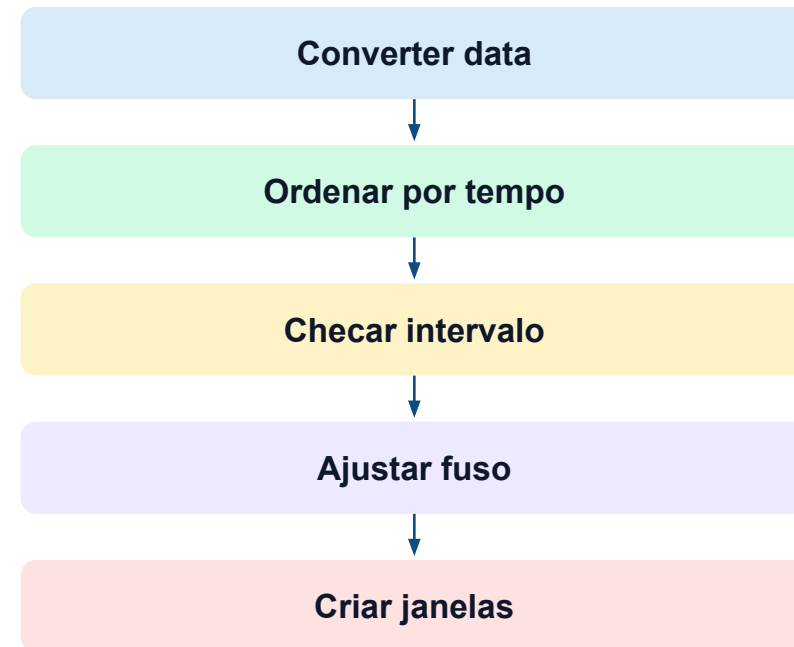
Timestamp e ordenação temporal

Tópico: Tempo como eixo da análise

Em bases de sensores, a ordem temporal é fundamental. Leituras fora de ordem, relógios desalinhados ou fusos incorretos distorcem médias, janelas e tendências. Antes de minerar, é necessário converter datas, ordenar registros e verificar intervalos entre medições.

[Link útil: Grafana time series](#)

Tempo



Amostragem e granularidade

Tópico: Frequência de leitura

A frequência de amostragem define quantas leituras são coletadas em determinado intervalo. Dados muito espaçados podem esconder eventos rápidos; dados muito frequentes podem gerar volume desnecessário. A granularidade precisa ser compatível com a decisão esperada.

Granularidade

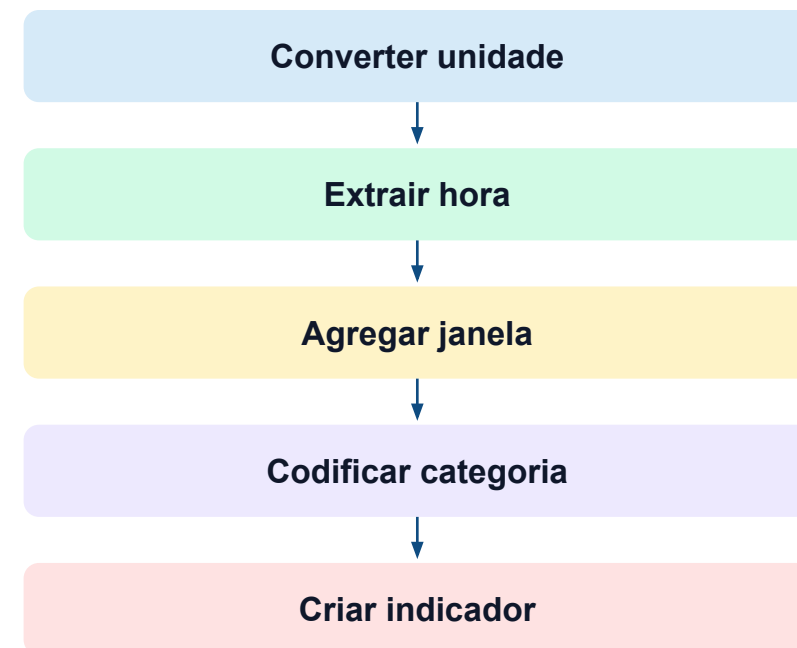


Transformação de dados

Tópico: Mudar representação para analisar melhor

Transformar dados significa criar uma representação mais útil para o algoritmo ou para a interpretação. Podemos converter unidades, extrair hora do dia, calcular médias por janela, transformar categorias em números e criar indicadores que resumem comportamento.

Transformação

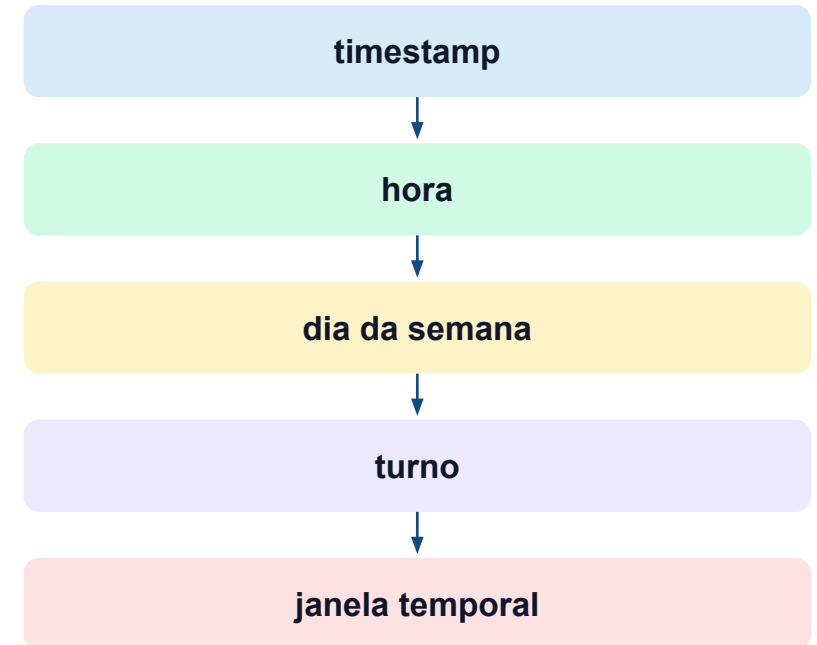


Transformações temporais

Tópico: Data e hora viram atributos

Datas podem ser transformadas em hora, dia da semana, turno, fim de semana, mês ou período letivo. Em IoT, esses atributos ajudam a explicar padrões: consumo maior à noite, temperatura elevada à tarde ou presença concentrada em horários de aula.

Atributos temporais

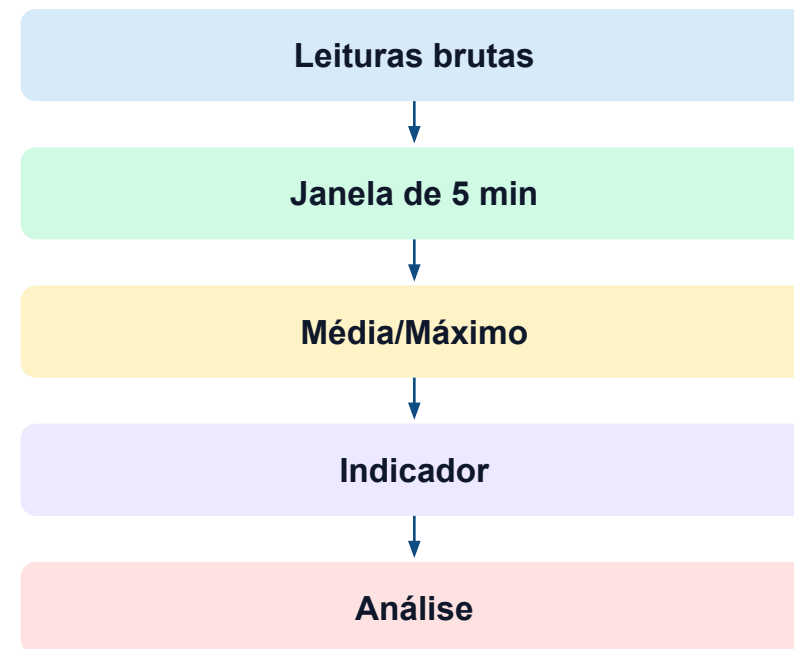


Agregação por janelas

Tópico: Média, máximo, mínimo e soma

Agregação resume várias leituras em um indicador por intervalo. Podemos calcular média de temperatura a cada cinco minutos, máximo de vibração por minuto ou consumo total por hora. A escolha do indicador deve preservar o comportamento importante para o problema.

Janelas



Criação de atributos

Tópico: Feature engineering

Atributos derivados podem revelar padrões que não aparecem diretamente nos valores brutos. Exemplos: diferença entre leitura atual e anterior, média móvel, variação percentual, tempo desde o último evento, quantidade de leituras inválidas ou indicador de sensor offline.

Feature engineering

- ✓ média móvel
- ✓ variação
- ✓ máximo por janela
- ✓ flag de ausência
- ✓ tempo desde evento
- ✓ leituras inválidas

Categorias e codificação

Tópico: Transformar texto em variável utilizável

Alguns campos são categóricos, como status do sensor, tipo de ambiente, turno ou estado do equipamento. Muitos algoritmos exigem números, então usamos codificação. O cuidado é não criar uma ordem artificial quando as categorias são apenas nomes.

Dados depois da limpeza

hora	temp	umid	status
08:00	23.1	62	ok
08:05	23.6	63	estim.
08:10	24.0	90	alerta
08:15	23.6	64	ok

ausentes tratados, duplicidade removida e unidade validada

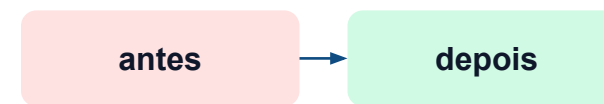
Normalização

Tópico: Colocar valores em intervalo comum

Normalização geralmente transforma valores para uma escala comum, como zero a um. Isso ajuda quando variáveis têm grandezas diferentes, por exemplo temperatura em graus, umidade em porcentagem e vibração em unidades muito maiores. Alguns algoritmos são sensíveis a essas diferenças.

[Link útil: MinMaxScaler](#)

Normalização (0 a 1)



valores no mesmo intervalo facilitam comparação visual e algoritmos

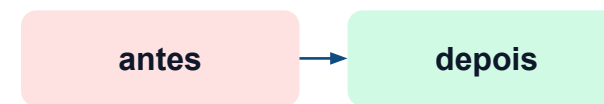
Padronização

Tópico: Média zero e desvio padrão um

Padronização transforma uma variável considerando sua média e seu desvio padrão. O resultado indica quantos desvios cada valor está acima ou abaixo da média. É muito usada antes de agrupamento, regressão e modelos que dependem de distância entre registros.

[Link útil: StandardScaler](#)

Padronização (z-score)



média 0 e desvio 1 ajudam modelos sensíveis à escala

Normalização ou padronização?

Tópico: Como escolher

Não existe uma regra única. Normalização é útil quando queremos limitar valores a um intervalo conhecido. Padronização é útil quando a distribuição e a distância em relação à média importam. Em ambos os casos, a transformação deve ser ajustada no treino e aplicada ao teste.

Escala

Normalização Min-Max

$$x' = (x - \text{mínimo}) / (\text{máximo} - \text{mínimo})$$

Padronização z-score

$$z = (x - \text{média}) / \text{desvio padrão}$$

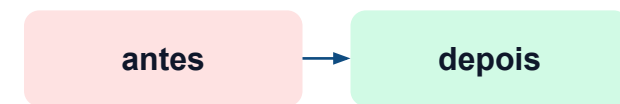
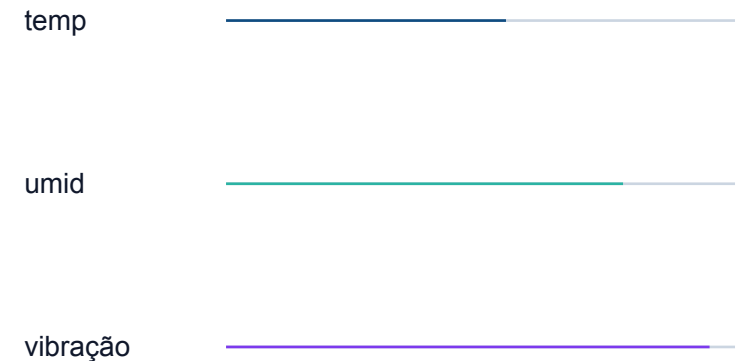
Use no conjunto de treino e aplique a mesma regra no teste.

Escalas em sensores heterogêneos

Tópico: Comparação entre grandezas diferentes

Projetos de IoT combinam sensores com escalas variadas. Temperatura, CO₂, vibração, luminosidade e consumo elétrico podem ter unidades e amplitudes muito diferentes. Sem ajuste de escala, algoritmos baseados em distância podem priorizar a variável de maior valor numérico.

Normalização (0 a 1)



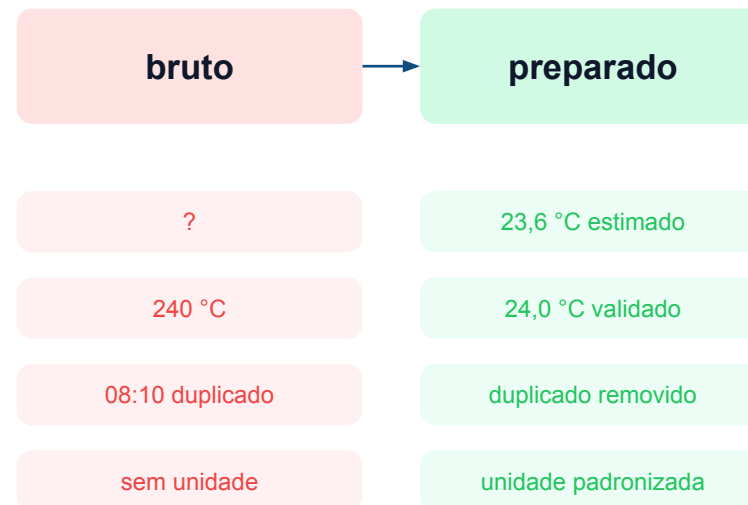
valores no mesmo intervalo facilitam comparação visual e algoritmos

Base antes e depois

Tópico: Comparar para validar

Após a limpeza e transformação, é importante comparar a base preparada com a base original. Essa comparação mostra o que mudou, quantos registros foram afetados e se o tratamento preservou o comportamento principal dos dados. Transparência evita conclusões frágeis.

Antes e depois



A preparação deve ser documentada para que outro grupo consiga reproduzir as decisões.

Registrar decisões de preparação

Tópico: Reprodutibilidade

Toda decisão deve ser registrada: critério de exclusão, método de preenchimento, filtro aplicado, unidade adotada, janela escolhida e fórmula de padronização. Em ciência de dados, reprodutibilidade significa que outra pessoa consegue executar os mesmos passos e obter resultado equivalente.

Registro

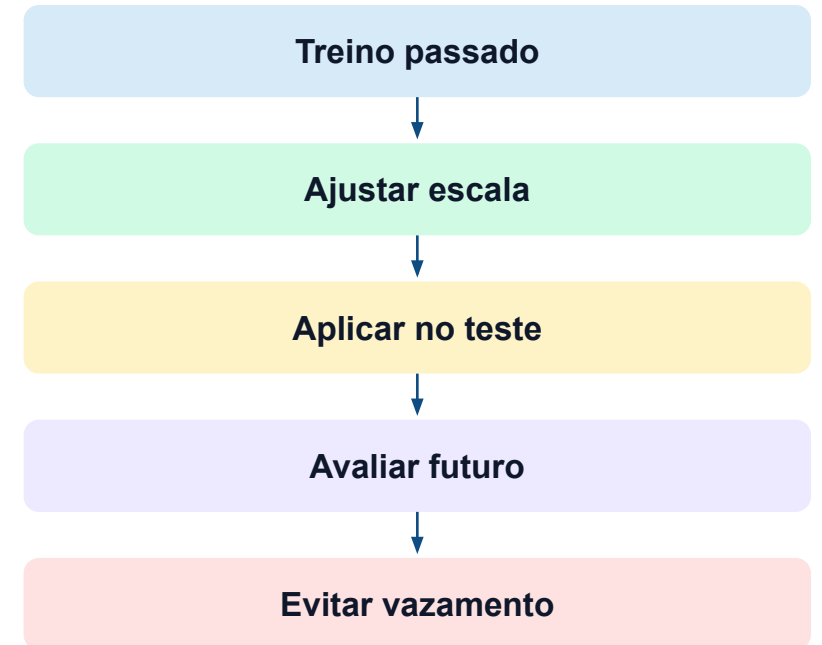
- ✓ critério de limpeza
- ✓ método de imputação
- ✓ janela temporal
- ✓ filtro de ruído
- ✓ escala usada
- ✓ versão da base

Cuidado com vazamento de dados

Tópico: Treino e teste

Vazamento ocorre quando informações do conjunto de teste entram no treinamento de forma indireta. Por exemplo, calcular média, mínimo ou máximo usando toda a base antes da divisão pode tornar o resultado artificialmente melhor. Em séries temporais, também devemos respeitar a ordem do tempo.

Treino/teste

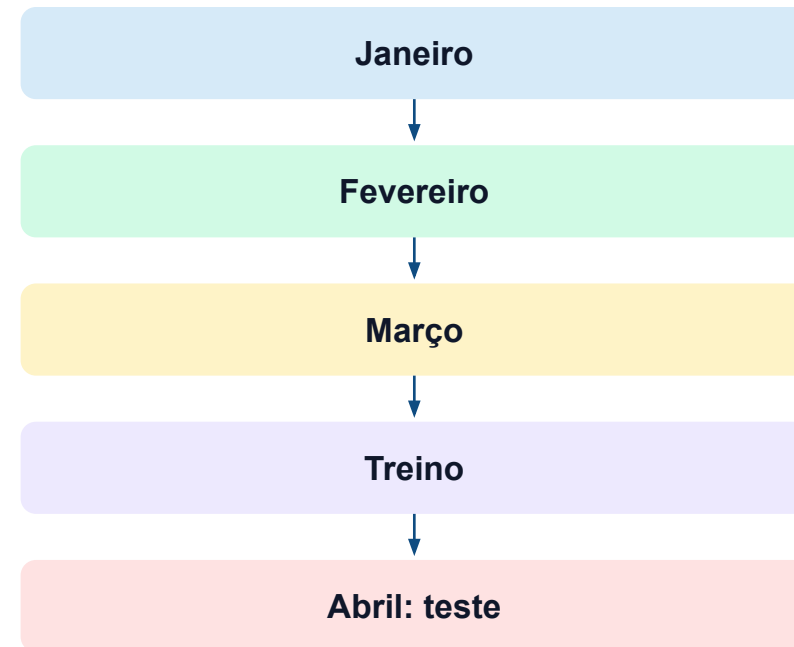


Divisão temporal da base

Tópico: Validação em séries temporais

Em muitos cenários de IoT, não devemos embaralhar registros aleatoriamente, pois o objetivo é prever ou detectar eventos futuros. Uma prática comum é treinar com períodos antigos e testar em períodos posteriores, simulando melhor o uso real da aplicação.

Validação temporal



Ferramentas gratuitas para prática

Tópico: Ambiente acadêmico

Para a aula prática, os alunos podem usar Google Colab com Python e pandas, Orange Data Mining para fluxo visual, planilhas para inspeção inicial e bases CSV simuladas. Essas opções são gratuitas ou possuem uso acadêmico acessível e não exigem infraestrutura complexa.

[Link útil: Orange Preprocess](#)

Ferramentas

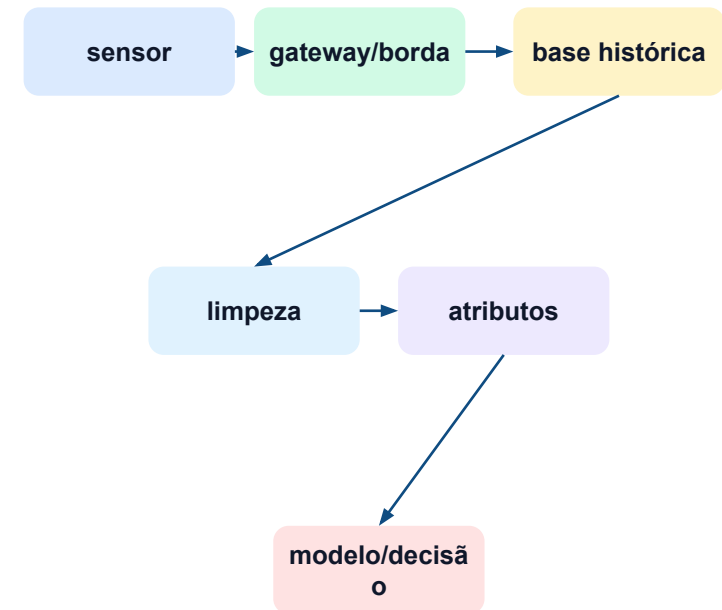
- ✓ Google Colab
- ✓ Python/pandas
- ✓ Orange Data Mining
- ✓ CSV simulado
- ✓ Scikit-learn
- ✓ gráficos simples

Prática guiada: cenário

Tópico: Sala inteligente

Considere uma sala com sensores de temperatura, umidade, presença e consumo de energia. O objetivo é preparar os dados para descobrir horários de desconforto térmico e possíveis desperdícios. A base terá valores ausentes, ruído, duplicidade e escalas diferentes.

IoT: do sensor ao modelo



Passo 1: criar ou obter o CSV

Tópico: Base inicial da prática

O professor pode fornecer um arquivo CSV simples ou os alunos podem simular dados. As colunas mínimas são timestamp, sensorId, temperatura, umidade, presença e consumo. Inclua alguns erros controlados para que a turma consiga praticar diagnóstico e tratamento.

CSV de exemplo

```
timestamp,sensorId,temp,umid,presenca,consumo
2026-09-01 08:00,sala1,23.1,62,1,0.8
2026-09-01 08:05,sala1,,63,1,0.9
2026-09-01 08:10,sala1,240,90,1,1.5
2026-09-01 08:10,sala1,240,90,1,1.5
```

Ambiente gratuito: Google Colab ou instalação local do Python.

Passo 2: carregar e inspecionar

Tópico: Primeiro contato com os dados

No pandas, carregamos o CSV e observamos dimensões, tipos das colunas e primeiras linhas. No Orange, usamos o widget File e Data Table. Essa etapa responde perguntas básicas: quantos registros existem, quais colunas faltam e quais formatos parecem incorretos.

Carregar no pandas

```
import pandas as pd
df = pd.read_csv('sensores.csv')
df.head()
df.info()
df.isna().sum()
df.describe()
```

Ambiente gratuito: Google Colab ou instalação local do Python.

Passo 3: diagnosticar problemas

Tópico: Encontrar falhas antes de corrigir

O diagnóstico deve localizar ausentes, duplicidades, valores fora de faixa e timestamps repetidos ou fora de ordem. Para sensores ambientais, podemos considerar temperatura plausível entre 0 °C e 50 °C em sala de aula, mas esse limite deve ser ajustado ao contexto.

Diagnóstico

```
df.duplicated().sum()
df['timestamp'] = pd.to_datetime(df['timestamp'])
df = df.sort_values('timestamp')
df.query('temp < 0 or temp > 50')
df['sensorId'].value_counts()
```

Ambiente gratuito: Google Colab ou instalação local do Python.

Passo 4: limpar ausentes e ruído

Tópico: Aplicar tratamento controlado

Uma solução simples é remover duplicados, marcar temperatura impossível como ausente e preencher lacunas por interpolação temporal. Depois, podemos calcular média móvel para suavizar pequenas oscilações. Cada passo deve ser feito com cuidado e registrado no relatório.

Limpeza prática

```
df = df.drop_duplicates()
df.loc[(df.temp < 0) | (df.temp > 50), 'temp'] =
pd.NA
df['temp'] = df['temp'].interpolate()
df['temp_mm3'] = df['temp'].rolling(3,
min_periods=1).mean()
df[['temp', 'temp_mm3']].head()
```

Ambiente gratuito: Google Colab ou instalação local do Python.

Passo 5: transformar e padronizar

Tópico: Preparar atributos para mineração

Depois da limpeza, criamos atributos como hora do dia, indicador de ambiente ocupado e consumo por presença. Para modelos sensíveis à escala, aplicamos padronização ou normalização. A transformação deve ser feita de modo consistente entre treino, teste e novos dados.

Transformação e escala

```
df['hora'] = df['timestamp'].dt.hour
df['ocupada'] = df['presenca'].astype(int)
cols = ['temp_mm3', 'umid', 'consumo']
from sklearn.preprocessing import StandardScaler
scaler = StandardScaler()
df[cols] = scaler.fit_transform(df[cols])
```

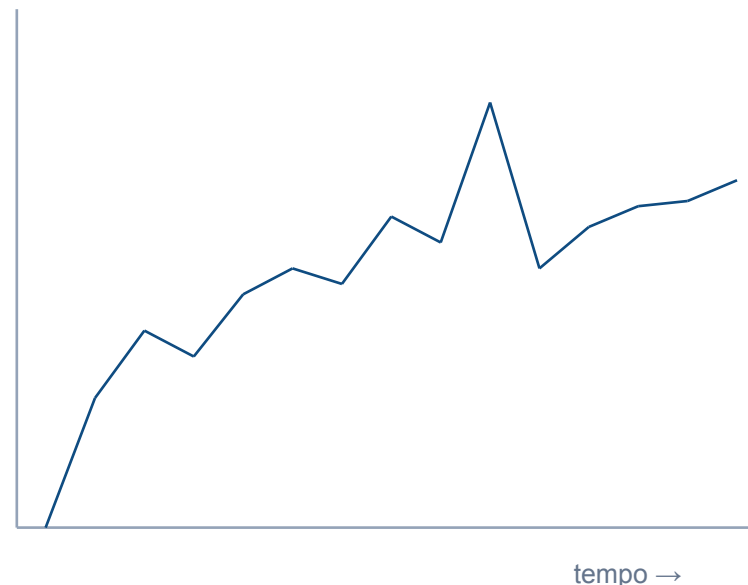
Ambiente gratuito: Google Colab ou instalação local do Python.

Passo 6: visualizar antes e depois

Tópico: Conferir se o tratamento faz sentido

A visualização ajuda a validar a preparação. Gráficos de linha mostram tendência e ruído; tabelas de resumo mostram ausentes e duplicados; histogramas revelam mudanças na distribuição. Se o gráfico final parecer artificial, a limpeza pode ter sido agressiva demais.

Ruído e picos



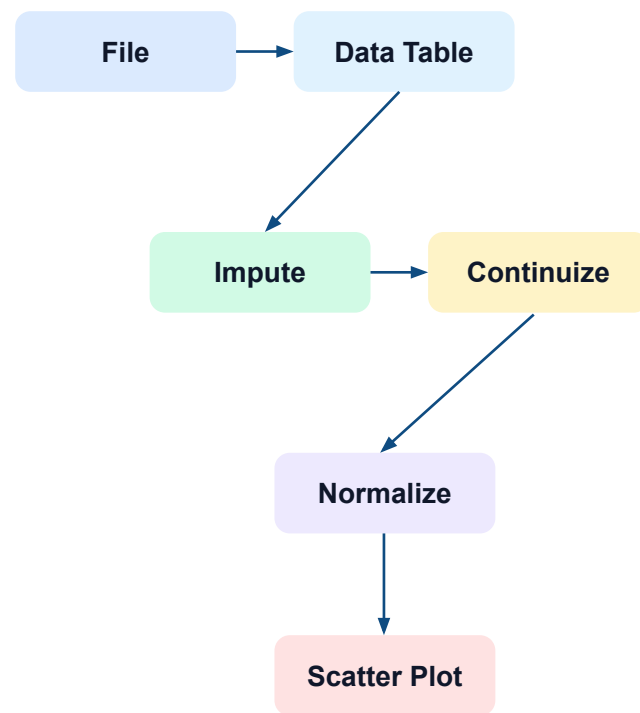
Alternativa sem programação: Orange

Tópico: Fluxo visual de pré-processamento

No Orange Data Mining, o aluno pode montar um fluxo com File, Data Table, Impute, Continuize, Normalize e Scatter Plot. A vantagem é enxergar cada etapa como um bloco. Isso ajuda iniciantes a compreender o processo antes de escrever código.

[Link útil: Orange Data Mining](#)

Orange Data Mining



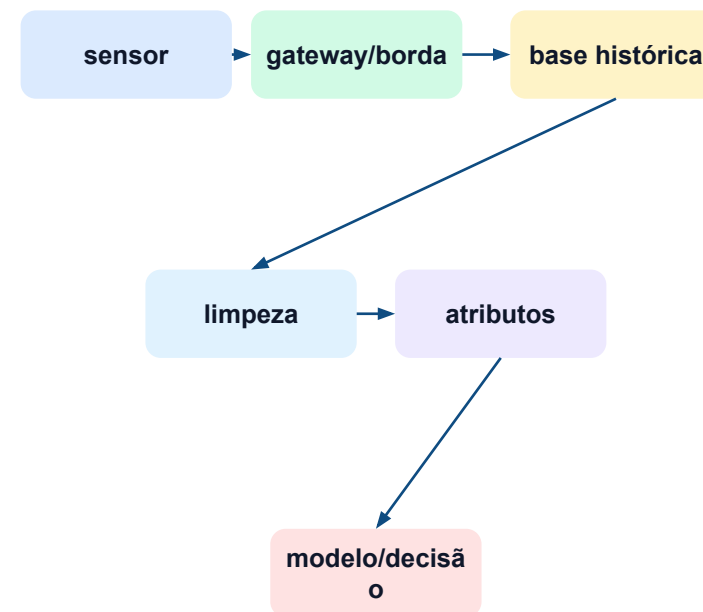
Fluxo visual: bom para iniciar sem programar e comparar dados antes/depois.

Aplicação: sala inteligente

Tópico: Da preparação à decisão

Depois do pré-processamento, a base pode indicar horários de sala ocupada com consumo alto, períodos de desconforto térmico ou falhas de sensor. O conhecimento aparece quando conectamos os padrões tratados com uma decisão prática, como ajustar climatização ou investigar equipamento.

IoT: do sensor ao modelo



Checklist de entrega da prática

Tópico: O que o aluno deve apresentar

A entrega deve conter o problema escolhido, descrição da base, diagnóstico dos problemas, tratamentos aplicados, justificativa das decisões, comparação antes e depois e uma conclusão. O foco avaliativo é a coerência entre problema, dados, preparação e interpretação final.

Entrega

- ✓ problema e objetivo
- ✓ base e colunas
- ✓ diagnóstico
- ✓ tratamentos
- ✓ justificativa
- ✓ antes/depois
- ✓ conclusão

Atividade em grupo

Tópico: Aplicação em sala

Cada grupo deve escolher um cenário de IoT, listar sensores, simular uma pequena base com pelo menos três problemas e propor um pipeline de pré-processamento. Depois, deve explicar quais tratamentos foram aplicados e como a base preparada ajudaria uma análise de mineração.

Atividade

- ✓ cenário IoT
- ✓ sensores
- ✓ base simulada
- ✓ 3 problemas
- ✓ pipeline
- ✓ decisão esperada

Erros comuns

Tópico: O que evitar

Erros frequentes incluem preencher ausentes sem investigar a causa, remover outliers que eram eventos reais, misturar unidades, ignorar timestamp, aplicar escala antes de dividir treino e teste, não registrar decisões e acreditar que o algoritmo resolverá problemas de coleta.

Evite

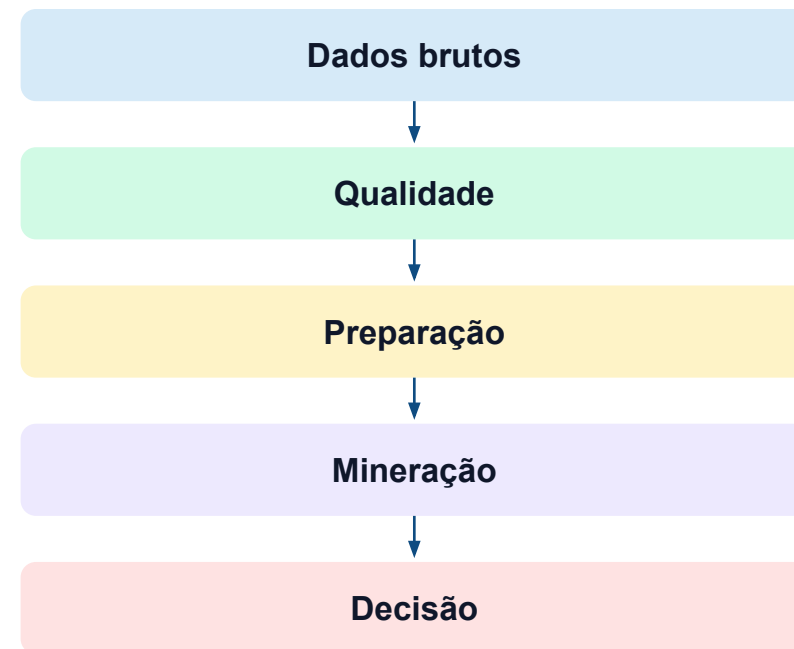
- ✓ não investigar ausências
- ✓ apagar eventos reais
- ✓ misturar unidades
- ✓ ignorar tempo
- ✓ vaziar dados
- ✓ não documentar

Síntese da aula

Tópico: Ideias principais

Pré-processamento transforma dados brutos de sensores em uma base mais confiável para mineração. Limpeza, tratamento de ruído, valores ausentes, transformação e padronização reduzem distorções e melhoram a interpretação. O algoritmo só produz conhecimento útil quando os dados foram preparados com critério.

Síntese



Referências e materiais recomendados

Tópico: Fontes para aprofundamento

Materiais da disciplina: KDD aplicado à Internet das Coisas; Fluxo e características dos dados de sensores; Comunicação e arquitetura IoT. Fontes complementares: pandas, scikit-learn, Orange Data Mining, NIST IoT, Grafana time series e artigo clássico de Fayyad, Piatetsky-Shapiro e Smyth sobre KDD.

Referências

- ✓ pandas: missing data
- ✓ scikit-learn: preprocessing
- ✓ Orange: Preprocess
- ✓ NIST: IoT
- ✓ Grafana: time series
- ✓ Fayyad et al.: KDD

Links nas anotações: pandas, scikit-learn, Orange, NIST, Grafana e KDD.